Projeto Integrador: Cadastro e Login

Funções, condicionais, repetição, arquivos e POO em um só programa

Aula 19 • 03/11/2026 • Linguagem de Programação Python (80h) • CETAM

Profa. Priscila Gonçalves

Visão geral da aula

Data

Terça-feira, 03/11/2026
Aula 19 • 4 tempos
Laboratório de informática

Habilidade técnica

Executar os conceitos iniciais de POO, com clareza e compreensão.

Projeto

Cadastro de usuário com senha, login e mensagem personalizada, persistidos em arquivo

Habilidade técnica

- Executar os conceitos iniciais de POO, com clareza e compreensão
- Integrar funções, condicionais, repetição, arquivos e POO
- Desenvolver programas funcionais com raciocínio lógico
- Trabalhar com criatividade, resiliência e colaboração

Objetos de conhecimento

- Introdução à Programação Orientada a Objetos
- Fundamentos de POO
- Integração de funções, condicionais, repetição e arquivos
- Depuração e testes em equipe

Roteiro dos 4 tempos

1º tempo

Integração de funções, condicionais, repetição, arquivos e POO

2º tempo

Programas funcionais, correção de erros em equipe e avaliação formativa

3º tempo

Alinhamento dos requisitos do projeto

4º tempo

Pair programming, depuração coletiva e combinados da AV4

1

1º tempo • Integração

Funções, condicionais, repetição, arquivos e POO

Retomada da Fase 5

- Aulas 15 e 16: funções, parâmetros, retorno e escopo
- Aula 17: arquivos .txt com with open()
- Aula 18: classes, objetos, __init__ e self
- Hoje: juntar tudo em um único programa
- Cada peça já foi vista; agora elas trabalham juntas

As cinco peças do projeto

Funções

Dividem o programa em tarefas

Decisões

Condicionais: login válido ou inválido

Repetição

Menu que volta até o usuário sair

Arquivos

Guardam os usuários entre execuções

POO

Classe Usuario com dados e ações

Como as peças se encaixam



O programa carrega os dados, repete o menu e salva as alterações.

Revisão: a classe Usuario

```
class Usuario:
    def __init__(self, nome, senha):
        self.nome = nome
        self.senha = senha

    def autenticar(self, senha):
        return self.senha == senha

    def mensagem(self):
        return f"Bem-vindo(a), {self.nome}!"
```

Revisão: arquivos

```
import os

def salvar_usuarios(usuarios):
    with open("usuarios.txt", "w", encoding="utf-8") as f:
        for u in usuarios:
            f.write(f"{u.nome};{u.senha}\n")

def carregar_usuarios():
    usuarios = []
    if os.path.exists("usuarios.txt"):
        with open("usuarios.txt", "r", encoding="utf-8") as f:
            for linha in f:
                nome, senha = linha.strip().split(";")
                usuarios.append(Usuario(nome, senha))
    return usuarios
```

Revisão: função com condicionais

```
def fazer_login(usuarios):
    nome = input("Nome: ").strip().title()
    senha = input("Senha: ")
    for u in usuarios:
        if u.nome == nome and u.autenticar(senha):
            print(u.mensagem())
            return True
    print("Nome ou senha incorretos")
    return False
```

Organização sugerida do projeto

- Classe Usuario no início do arquivo
- Funções de arquivo: salvar_usuarios e carregar_usuarios
- Funções do sistema: cadastrar_usuario e fazer_login
- Programa principal no final, com o menu
- Comentários dividindo cada bloco

2

2º tempo • Desenvolvimento e formativa

Raciocínio lógico, criatividade, resiliência e colaboração

Objetivo do 2º tempo

- Desenvolver programas funcionais em Python
- Usar raciocínio lógico para planejar cada etapa
- Usar criatividade nas soluções e nas mensagens
- Corrigir erros com resiliência e colaboração em equipe
- Avançar com o projeto, não só consertar bugs

Avaliação formativa

- Avaliação durante o processo, sem foco só na nota
- Acompanhamento do docente nas equipes
- Registro do progresso de cada dupla
- Devolutiva imediata para ajustar o caminho
- Errar e corrigir faz parte da avaliação

Ficha de acompanhamento

Planejamento

A equipe sabe o que vai fazer e em que ordem?

Funcionamento

O que já roda corretamente?

Colaboração

Todos participam e se ajudam?

Resiliência

Como a equipe lida com erros e frustrações?

Como corrigir erros em equipe

- Leia a mensagem de erro em voz alta
- Identifique o arquivo e a linha
- Formule uma hipótese antes de mexer no código
- Teste uma mudança por vez
- Se travar, peça ajuda a outra dupla antes de chamar a professora

Bug 1: esqueceu o self

```
class Usuario:
    def __init__(self, nome, senha):
        self.nome = nome
        senha = senha       # faltou self.

    def autenticar(self, senha):
        return self.senha == senha
        # AttributeError: sem self.senha
```

- O atributo nunca foi criado
- Correção: self.senha = senha
- Leia a mensagem de erro

Bug 2: o cadastro apaga usuários

```
# Errado: "w" apaga os usuários anteriores
with open("usuarios.txt", "w", encoding="utf-8") as f:
    f.write(f"{nome};{senha}\n")

# Correto: "a" acrescenta ao final
with open("usuarios.txt", "a", encoding="utf-8") as f:
    f.write(f"{nome};{senha}\n")
```

3

3º tempo • Requisitos

Alinhamento com a turma

Requisitos do projeto

- Cadastro de usuário com senha
- Login com nome e senha
- Mensagem personalizada após o login
- Tudo persistido em arquivo
- Alinhamento: vamos combinar o que é obrigatório e o que é opcional

Os quatro requisitos

Cadastro

Pede nome e senha, valida a senha e cria o Usuario.

Login

Compara nome e senha com os usuários cadastrados.

Mensagem

Cumprimenta pelo nome, de forma personalizada.

Persistência

Salva e carrega os usuários do arquivo.

Formato do arquivo usuarios.txt

```
Ana;abc123
Caio;xyz789
Bia;senha4
```

- Um usuário por linha
- nome;senha
- Dados fictícios de teste

Fluxo do sistema



Cada opção do menu usa funções e o objeto Usuario.

Esqueleto do programa principal

```
usuarios = carregar_usuarios()
while True:
    print("1-Cadastrar 2-Login 0-Sair")
    op = input("Opção: ")
    if op == "0":
        break
    elif op == "1":
        cadastrar_usuario(usuarios)
    elif op == "2":
        fazer_login(usuarios)
    else:
        print("Opção inválida")
salvar_usuarios(usuarios)
```

Esqueleto: cadastrar_usuario

```
def cadastrar_usuario(usuarios):
    nome = input("Nome: ").strip().title()
    senha = input("Senha: ")
    while len(senha) < 4:
        print("Mínimo de 4 caracteres")
        senha = input("Senha: ")
    usuarios.append(Usuario(nome, senha))
    salvar_usuarios(usuarios)
    print("Cadastro realizado!")
```

Critérios de aceite

- Cadastro funciona e rejeita senha curta
- Login correto mostra a mensagem personalizada
- Login errado mostra aviso e volta ao menu
- Os dados continuam após fechar e abrir o programa
- Menu repete até a opção 0

Casos de teste sugeridos

- Cadastrar um usuário e fazer login com os mesmos dados
- Login com senha errada
- Login com usuário que não existe
- Cadastrar dois usuários e conferir o arquivo
- Fechar e abrir o programa: o login ainda funciona

Cuidados com senhas

- Use apenas dados fictícios: nunca senhas reais
- Guardar senha em texto puro é só para estudo
- Em sistemas reais, senhas são guardadas com hash
- Não imprima a senha na tela
- Não compartilhe o arquivo de teste fora da turma

4

4º tempo • Pair programming

Depuração coletiva e combinados da AV4

Pair programming

- Desenvolvimento em duplas
- Piloto: digita o código
- Copiloto: orienta, confere e pensa na próxima etapa
- Alternância de papéis a cada 10 minutos
- Os dois explicam o que foi feito

Piloto × copiloto

Piloto

Escreve o código e executa os testes.
Fala em voz alta o que está fazendo.

Copiloto

Lê o código, aponta erros, sugere ideias e consulta o roteiro.

Roteiro de trabalho da dupla

- 1. Releia os requisitos e escolha a próxima etapa
- 2. Piloto codifica por 10 minutos
- 3. Troquem de papel e testem
- 4. Registrem o que funciona e o que falta
- 5. Repitam o ciclo até concluir

Depuração coletiva

- Testes cruzados: cada dupla testa o programa de outra
- Quem testa tenta cadastros, logins certos e errados
- Registrar o que quebrou, sem culpar ninguém
- Devolver os bugs à dupla autora
- A dupla autora corrige e testa de novo

Registro de bugs

O que aconteceu

Descreva o erro e a mensagem

Onde

Função, linha ou opção do menu

Como reproduzir

Passos para o erro aparecer

Correção

O que foi feito e quem corrigiu

Ajustes finais

- Corrigir os bugs encontrados nos testes cruzados
- Conferir os requisitos um a um
- Revisar nomes, comentários e mensagens ao usuário
- Conferir o arquivo de dados e o tratamento de erros
- Rodar o programa do início ao fim sem travar

Preparação da apresentação

- Mostrar o programa funcionando: cadastro, login e persistência
- Explicar a divisão em classe, funções e arquivo
- Dizer quem fez cada parte e o que aprendeu com os erros
- Apresentar o registro de bugs
- Treinar o tempo e combinar quem fala cada trecho

Combinados da AV4

- Apresentação em duplas, com demonstração do programa
- Os dois integrantes explicam o código
- Critérios: funcionamento, lógica, organização, colaboração e apresentação
- Entrega do arquivo .py e do arquivo de dados de teste
- Data, tempo e ordem das apresentações combinados com a turma

Fechamento

- Hoje: integração de funções, condicionais, repetição, arquivos e POO
- Requisitos alinhados: cadastro, login e mensagem em arquivo
- Pair programming, depuração coletiva e registro de bugs
- Combinados da AV4 feitos. Boa apresentação!